IntentChecker: Validating Developer Intentions in Solidity via Intent Annotation to Mitigate Numeric Logic error

Inseong Jeon[1], Sundeuk Kim[1], Hyunwoo Kim[1], Hoh Peter In[1*]

[1*]Department of Computer Science, Korea University, 145, Anam-ro, Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr;
khw0809@korea.ac.kr;

Abstract

Logic errors in smart contracts—where program behavior diverges from developer intent—are among the most challenging vulnerabilities to detect. Unlike well-defined patterns such as reentrancy or integer overflow, logic errors require knowledge of developer intent, which is absent from the code itself. Existing approaches, including static analyzers, dynamic testing, and LLM-based methods, cannot reliably identify such errors without explicit intent representation. This paper presents IntentChecker, a tool that enables developers to specify and validate their intentions for numeric operations during smart contract development. IntentChecker introduces an intent annotation model with two annotation types: `@During` annotations specify intended relationships at specific program points within a function, while `@Post` annotations specify intended relationships after function execution. The underlying analysis uses interval-domain abstract interpretation to compute value ranges at each program point and check whether they satisfy the specified intentions. We evaluated IntentChecker on real-world smart contracts, demonstrating its effectiveness in validating developer intentions and providing immediate feedback during development.

Keywords: Smart Contract Development, Solidity, Numeric Logic Error, Intent Annotations, Abstract Interpretation, Developer Tools

1 Introduction

Smart contracts are immutable once deployed, making security assurance during development critical. Among vulnerability types, logic errors present a distinct challenge: they arise when program behavior diverges from developer intent, a discrepancy that is inherently invisible to compilers and external reviewers alike (Soud et al., 2024).

A recent empirical study (Chaliasos et al., 2024) confirms this challenge in practice. Practitioners identified logic errors as the most difficult vulnerability type to detect, reporting that existing security tools are least effective against them. This gap exists because current approaches—static analyzers, dynamic testing, and even recent LLM-based methods—predominantly target well-defined vulnerability patterns. Logic errors, however, require knowledge of developer intent, which is absent from the code itself. Various research efforts have attempted to detect specific logic errors, but without explicit intent representation, they must rely on predefined detection patterns. Such patterns can identify violations of specific rules, but not actual deviations from intended behavior. Consequently, even when these tools report no issues, developers cannot be certain that their code behaves as intended.

(Chaliasos et al., 2024) also provides insight into developer needs. Developers expressed a preference for development-integrated tools such as IDE extensions and linters over standalone analyzers. Moreover, when asked about the most valued properties of security tools, developers prioritized low false negatives (92.3%) over low false positives (50%), indicating a strong preference for tools that do not miss real issues, even at the cost of occasional false alarms. This suggests a need for development-phase tools that can provide sound guarantees about code behavior. To determine where such intent specification is most needed, we analyzed 1,023 non-trivial functions from DAppSCAN-source (Zheng et al., 2024) (see Table **??**). Numeric variable types appear in 82% of these functions, with `uint` alone used in 78%. Since numeric computations form the core of smart contract business logic —handling token transfers, fee calculations, and balance updates—explicitly specifying developer intent for these operations becomes essential.

Based on these observations, this paper presents IntentChecker, a tool that enables developers to specify and validate their intentions for numeric operations during the development phase. IntentChecker introduces an intent annotation model consisting of two annotation types: `@During` annotations specify intended relationships that should hold at specific program points within a function, while `@Post` annotations specify intended relationships that should hold after the function completes execution. Developers write these annotations alongside their code, and IntentChecker validates whether the program's abstract execution satisfies these specified intentions.

The underlying value analysis is performed using interval-domain abstract interpretation, building upon our previous work on SolQDebug (Jeon et al., 2025), an interactive debugger for Solidity. Through `@Debugging` annotations, developers specify value ranges for variables, and the analyzer computes abstract values at each program point. IntentChecker extends this capability by checking whether these computed values satisfy the developer-specified intent annotations, providing immediate feedback during development rather than requiring completed code.

This paper makes the following contributions:

- We present an intent annotation model that enables developers to express intended numeric relationships at both statement-level (`@During`) and function-level (`@Post`) granularity.
- We develop validation algorithms for each annotation rule that leverage interval-domain abstract interpretation to check whether program behavior satisfies specified intentions.
- We evaluate IntentChecker on real-world smart contracts, demonstrating its effectiveness in validating developer intentions during the development phase.

The remainder of this paper is organized as follows. Section 2 provides background on numeric logic errors and their classification by execution semantics. Section 3 describes the design of IntentChecker, including its intent model and validation engine. Section 4 presents our evaluation results. Section 5 discusses limitations and future work. Section 6 reviews related work, and Section 7 concludes.

2 Background

2.1 Numeric Logic Errors in Solidity

Recent research has identified various numeric-related weakness in smart contracts (Chen et al., 2025; Sun et al., 2024; Soud et al., 2024; Zhang et al., 2023; Zhou et al., 2023). Building on these findings, numeric logic errors can be defined as follows:

Definition 1 (Numeric Logic Error) Bugs that allow transactions to complete successfully without compile-time or runtime exceptions, but violate the developer's intended numerical or state relationships during function execution.

These errors can manifest in different execution contexts. Single-transaction errors occur within the execution of a single function call—where computation, function calls, and output generation happen in one atomic sequence. In contrast, multiple-transaction errors only emerge through sequences of function calls, requiring analysis of inter-function dependencies and contract-level invariants. Examples of multiple-transaction errors include Risky First Deposit and Price Oracle Manipulation (Sun et al., 2024; Zhang et al., 2023), where attackers exploit state changes across multiple function invocations.

This paper focuses on single-transaction function execution, as this aligns with the development-phase validation model where developers can immediately verify individual function behaviors against their intentions. The representative single-transaction numeric logic error patterns identified in prior work are summarized below:

- **Div In Path** (Chen et al., 2025): Division results used in conditions can lead to unexpected control flow due to integer truncation.
- **Operator Order Issue** (Chen et al., 2025): Arithmetic order matters: `a/b*c` $\neq$ `a*c/b` due to intermediate truncation.
- **Exchange Problem** (Chen et al., 2025): Rounding errors in exchange calculations may result in zero output despite valid input, or receiving tokens without payment.

- **Precision Loss Trend** (Chen et al., 2025): Integer division's inherent truncation tends to favor one side, potentially misaligning with protocol intent.
- **Minor Amount Retention** (Chen et al., 2025): Integer division in distributions leaves remainders unallocated to any recipient, causing permanent lock.
- **Erroneous Accounting** (Sun et al., 2024; Zhang et al., 2023): Incorrect implementation of business model formulas introduces errors that accumulate into substantial losses. Representative sub-patterns include:
  - *Wrong Interest Rate Order*: Balance calculations using outdated or prematurely updated interest rates yield incorrect results.
  - *Wrong Checkpoint Order*: Incorrect ordering between state updates and checkpoints leads to reward miscalculation.
- **Greedy Contract** (Soud et al., 2024): Contract can receive funds but lacks proper logic to withdraw them, causing permanent lock.

Despite their diversity, these error patterns share a common root: the actual behavior diverges from what the developer intended.

2.2 Motivation for Intent-Based Validation

Existing approaches to detecting numeric logic errors—whether pattern-based static analyzers, formal verification tools, or LLM-based methods—fundamentally analyze code without access to the developer's actual intentions. Since logic errors arise when program behavior diverges from developer intent, and this intent exists only in the developer's mind, these approaches cannot reliably distinguish between intended behavior and unintended bugs. To bridge this gap, developers must be able to explicitly specify their intended numeric behaviors. The key question then becomes: how can we systematically represent numeric intentions within a single function execution?

Numeric variables flow through three distinct stages during function execution, and developers may have specific intentions at each stage:

- **COMPUTE**: Developers intend that expressions yield results within expected ranges, and that statements modify variables as intended (e.g., increasing, decreasing, or remaining unchanged).
- **TRANSFER**: Developers intend that function calls receive arguments satisfying certain constraints, ensuring that called functions operate on valid inputs.
- **OUTPUT**: Developers intend that functions return expected values and update state variables correctly, reflecting the intended effects of the computation.

This classification provides a foundation for designing an intent specification mechanism that can address errors at each stage of variable flow. Note that error patterns do not map strictly to a single stage—for instance, an operator order issue during computation may manifest as an incorrect argument passed to a transfer function. The intent system accommodates this by allowing developers to specify constraints at any relevant stage.

2.3 IntentChecker Overview

Based on the variable flow classification, we propose IntentChecker, a framework that enables developers to declare their intended numeric behaviors and validates them through static analysis. IntentChecker consists of two core components: an *Intent Model* for specifying developer intentions, and a *Validation Engine* for checking whether code behavior conforms to those intentions. The intent model provides two annotation types aligned with the variable flow stages:

- `@During`: Specifies intended relationships at specific program points within a function, covering COMPUTE and TRANSFER stages.
- `@Post`: Specifies intended relationships after function execution, covering OUTPUT stage and function-level invariants.

Developers express these intentions through annotations embedded in comments, allowing intent specifications to coexist with existing code without modification.

The validation engine builds upon SolQDebug (Jeon et al., 2025), a line-by-line debugging module that performs abstract interpretation over the interval domain, to compute actual value ranges of variables and compare them against the declared intent. The validation result is deterministically categorized into one of three states:

$$\textbf{Satisfied} \iff I_{\text{actual}} \subseteq I_{\text{intent}}$$

$$\textbf{Violated} \iff I_{\text{actual}} \cap I_{\text{intent}} = \varnothing$$

$$\textbf{Warning} \iff I_{\text{actual}} \not\subseteq I_{\text{intent}} \ \wedge \ I_{\text{actual}} \cap I_{\text{intent}} \neq \varnothing$$

If neither fully satisfied nor clearly violated, the system reports `Warning`, along with a risk score on a 0–10 scale quantifying the severity of the potential violation and the specific sub-intervals where violation may occur. Unlike warnings from pattern-based static analyzers that suggest likely bugs, IntentChecker's `Warning` indicates partial satisfaction—the intent holds for some input values but not others within the specified range.

SolQDebug was originally designed as a source-level Solidity debugger for single-transaction, single-contract analysis. IntentChecker extends this foundation to support multiple contracts defined within the same source file, with commonly used libraries pre-analyzed and their semantics imported during validation.

The detailed design of the intent model and validation algorithms are presented in Section 3.

3 The Design of IntentChecker

3.1 System Architecture

Figure 1 illustrates the overall architecture of IntentChecker.

Input. IntentChecker receives three types of input:

- **Statement**—individual Solidity statements provided incrementally as the developer writes code.

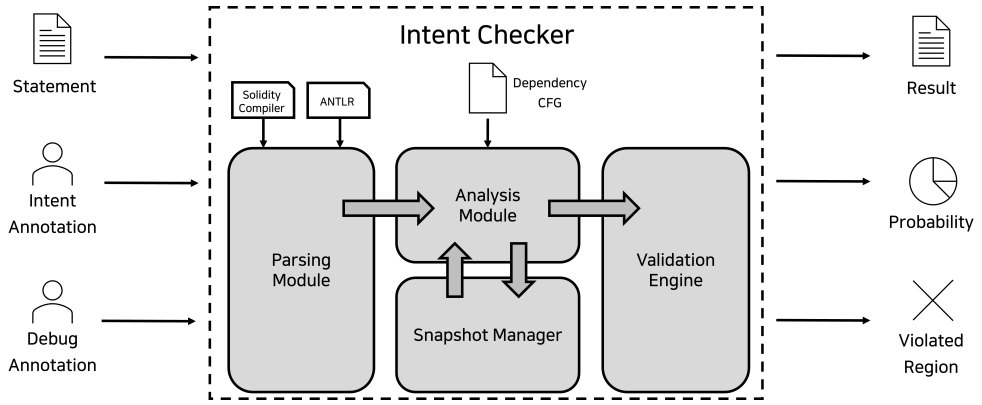

Fig. 1: System architecture of IntentChecker.

- **Intent Annotation**—`@During` and `@Post` specifications expressing the developer's intended numerical relationships.
- **Debug Annotation**—variable initialization directives such as `@StateVar` and `@LocalVar` that provide concrete or symbolic values for analysis.

Output. For each intent annotation, IntentChecker produces:

- **Result**—a validation state of `Satisfied`, `Violated`, or `Warning`.
- **Risk Score**—a 0–10 scale score: 0.0 for `Satisfied`, 10.0 for `Violated`, and 0.1–9.9 for `Warning` reflecting the severity of the potential violation.
- **Violated Region**—the specific sub-intervals where the intent may be violated, helping developers identify problematic input ranges.

Internal Modules. The system builds upon SolQDebug (Jeon et al., 2025), which provides the Parsing Module, Analysis Module, and Snapshot Manager. The Parsing Module uses ANTLR for syntax analysis and the Solidity compiler for semantic validation. The Analysis Module performs incremental abstract interpretation over the interval domain, updating the control-flow graph and variable states as each statement is processed. When a `using` directive imports an external library, the Analysis Module loads the corresponding Dependency CFG—a pre-analyzed representation of the library's functions—enabling accurate modeling of library calls without re-parsing the library source. When a debug annotation is received, the Snapshot Manager preserves the current analysis state, allowing the system to apply temporary variable bindings, re-analyze affected code paths, and restore the original state afterward.

The Validation Engine is IntentChecker's core contribution. It receives the computed variable intervals from the Analysis Module along with the parsed intent annotations. For each annotation, it evaluates whether the declared intent holds by comparing the actual intervals against the specified constraints, producing the validation result, risk score, and violated regions as described above.

3.2 Motivating Examples

This section illustrates how IntentChecker addresses representative numeric logic errors from each variable flow stage introduced in Section 2. For each example, we show the vulnerable code pattern and demonstrate how developers can express their intentions through annotations.

3.2.1 COMPUTE: Precision Loss in Arithmetic

Listing 1: Example: Precision loss in fee calculation

```
// Vulnerable: division before multiplication causes precision loss
uint256 fee = amount / 100 * feeRate;

// Developer's intent: fee should equal the mathematically correct result
// @During fee == amount * feeRate / 100
```

3.2.2 TRANSFER: Function Argument Validation

Listing 2: Example: Ensuring non-zero transfer amounts

```
// Vulnerable: integer division may result in zero transfer
for (uint i = 0; i < recipients.length; i++) {
    uint256 share = totalAmount / recipients.length;
    // @During transfer.arg[0] > 0
    recipients[i].transfer(share);
}
```

3.2.3 OUTPUT: State Consistency After Execution

Listing 3: Example: Verifying balance consistency

```
function withdraw(uint256 amount) external {
    require(balances[msg.sender] >= amount);
    balances[msg.sender] -= amount;
    // @Post balances[msg.sender](Entry > Exit)
    msg.sender.transfer(amount);
}
```

These examples demonstrate how IntentChecker enables developers to explicitly specify their numerical intentions. The complete intent model supporting these annotations is defined in the following section.

3.3 Intent Model

This section defines the syntax and semantics of intent annotations. While Section 2 introduced the conceptual foundation—`@During` for COMPUTE and TRANSFER stages, `@Post` for OUTPUT stage—here we specify the concrete grammar and meaning of each annotation construct.

Annotation Syntax Overview. Figure 2 presents the grammar of intent annotations. Intent annotations are embedded in single-line comments, allowing them to coexist with Solidity code without affecting compilation. Each annotation begins with

```
duringIntent   →  // @During duringClause (logicOp duringClause)*
postIntent     →  // @Post postClause (logicOp postClause)*

duringClause   →  intentValue (before relOp after)
               |  intentValue (assign relOp current)
               |  identifier .arg[ n ] relOp intentValue
               |  commonClause

postClause     →  intentValue (entry relOp exit)
               |  commonClause

commonClause   →  returnExpression relOp intentValue
               |  return[n] relOp intentValue
               |  intentValue relOp PercentOf( intentValue , p )
               |  intentValue relOp ceil( intentValue , d )
               |  intentValue relOp floor( intentValue , d )
               |  intentValue relOp intentValue
               |  intentValue => intentValue

intentValue    →  arithExpr
arithExpr      →  arithExpr (+ | -) arithTerm | arithTerm
arithTerm      →  arithTerm (* | / | %) arithFactor | arithFactor
arithFactor    →  number | [ number , number ] | varRef | ( arithExpr )
varRef         →  identifier subAccess*
subAccess      →  . identifier | [ expr ]

logicOp        →  && | ||
relOp          →  < | > | <= | >= | == | != | in | not in
```

Fig. 2: Grammar of intent annotations.

a marker (`@During` or `@Post`) followed by one or more clauses. Multiple clauses can be combined using logical operators: `&&` (conjunction) requires all clauses to hold, while `|`— (disjunction) requires at least one clause to hold.

Intent values within clauses support arithmetic expressions using standard operators (`+`, `-`, `*`, `/`, `%`), variable references with member and index access (e.g., `balances[msg.sender]`), numeric literals, and inline intervals written as `[a, b]` to represent value ranges directly. Relational operators include `<`, `>`, `<=`, `>=`, `==`, `!=`, as well as `in` and `not in` for interval membership tests.

@During Annotation. The `@During` annotation specifies intended relationships at specific program points within a function, validating developer intentions during the COMPUTE and TRANSFER stages of variable flow. It supports the following clause types:

- `x(before relOp after)`—compares the value of variable `x` immediately before and after an assignment statement. This validates that assignments modify variables as intended (e.g., `balance(before > after)` asserts that the balance decreases).
- `x(assign relOp current)`—compares the value being assigned to `x` against its current value. This is useful for validating incremental updates (e.g., `counter(assign == current + 1)`).

- `func.arg[n] relOp value`—validates the n-th argument passed to a function call. This covers the TRANSFER stage, ensuring that function calls receive arguments satisfying specified constraints before execution proceeds.

@Post Annotation. The `@Post` annotation specifies intended relationships that should hold after function execution completes, validating the OUTPUT stage of variable flow. It supports the following clause types:

- `x(entry relOp exit)`—compares the value of variable `x` at function entry against its value at function exit. This validates function-level effects on state variables (e.g., `totalSupply(entry == exit)` asserts that total supply remains unchanged).

Common Clauses. Both `@During` and `@Post` annotations share the following clause types:

- `returnExpression relOp value`—validates the expression being returned against a specified value or range.
- `return[n] relOp value`—for functions returning multiple values (tuples), validates the n-th return value.
- `x relOp PercentOf(y, p)`—validates that `x` equals `p` percent of `y`, accounting for integer arithmetic. This is particularly useful for fee calculations and token distributions.
- `x relOp ceil(y, d)` and `x relOp floor(y, d)`—validate rounding behavior, where `d` specifies the divisor. These clauses help developers express intended rounding directions in division operations.
- `x relOp y`—general relational comparison between two intent values.
- `x => y`—implication clause, asserting that if condition `x` holds, then condition `y` must also hold. This enables conditional intent specifications.

3.4 Validation Engine

The Validation Engine validates developer-specified intent annotations against actual program behavior computed through abstract interpretation. Given the CFG-based interval analysis from Section **??**, the engine evaluates each annotation clause and determines whether the intended relationship holds across all possible execution states.

To perform validation, the engine captures variable environments at specific program points:

- *before_env*: variable state immediately before a statement executes
- *after_env*: variable state immediately after a statement executes (referred to as *current* in assign clauses)
- *assign_env*: the value being assigned in an assignment statement
- *entry_env*: variable state at function entry
- *exit_env*: variable state at function exit, computed by joining all normal termination paths

3.4.1 Validation Targets

@During Validation. The `@During` annotation is validated immediately after the associated statement executes:

- `var(before op after)`: compares *before_env* against *after_env* for variable `var`.
- `var(assign op current)`: compares *assign_env* against the current value of `var`.
- `func.arg[n] op value`: validates the n-th argument passed to the function call.

@Post Validation. The `@Post` annotation is validated when function execution completes:

- `var(entry op exit)`: compares *entry_env* against *exit_env* for variable `var`.

Common Clauses. The following clauses can be used in both `@During` and `@Post` annotations:

- `returnExpression op value`: compares the return value against the evaluated `value`.
- `return[n] op value`: compares the n-th element of a tuple return value against `value`.
- exp_1 `op PercentOf(`exp_2`, p)`: compares exp_1 against $p\%$ of exp_2.
- exp_1 `op ceil(`exp_2`, d)` and exp_1 `op floor(`exp_2`, d)`: compares exp_1 against exp_2 rounded up or down to the nearest multiple of `d`.
- exp_1 `op` exp_2: general comparison between two expressions.

3.4.2 State Determination and Risk Scoring

The engine compares interval-domain values and produces one of three possible outcomes:

- *satisfied*: the condition holds for all values in the intervals.
- *violated*: the condition fails for all values in the intervals.
- *warning*: the condition holds for some values but not others.

Each result is mapped to a *risk score* on a 0–10 scale. A score of 0.0 indicates a fully satisfied condition, while 10.0 indicates a definite violation. For *warning* results, the score reflects the severity of the potential violation. The engine classifies warnings into three risk types based on the structure of the violated region: type 1 (single-direction, minor) maps to scores 0.1–3.3; type 2 (single-direction violation) maps to 3.4–6.6; and type 3 (bidirectional violation or equality operators) maps to 6.7–9.9. Within each band, the score is computed as $base + (1 - p_{true}) \times span$, where p_{true} is the probability that the condition holds under a uniform distribution assumption.

Clause-to-Interval Mapping. Table 2 shows how each clause type maps to the interval comparison inputs L and R in Algorithm 1.

Interval Comparison Algorithm. Algorithm 1 presents the interval comparison procedure. Given two integer intervals $L = [l_{\min}, l_{\max}]$ and $R = [r_{\min}, r_{\max}]$ with an operator op, the algorithm partitions L into three regions under a uniform distribution assumption over both intervals:

Table 2: Mapping of clause types to interval comparison inputs. Here, exp_1 and exp_2 denote evaluated intent expressions; *return value* is the value returned by the current return statement; *return value*$[n]$ is the n-th element when the function returns a tuple.

Clause Type	L	R
`var(before op after)`	*before_env*[`var`]	*after_env*[`var`]
`var(assign op current)`	*assign_env*[`var`]	*after_env*[`var`]
`func.arg[n] op value`	`arg[n]`	`value`
`var(entry op exit)`	*entry_env*[`var`]	*exit_env*[`var`]
`returnExpression op value`	*return value*	`value`
`return[n] op value`	*return value*$[n]$	`value`
exp_1 `op` exp_2	exp_1	exp_2

Table 3: Per-operator satisfied/violated conditions and region sizes. All size expressions are clamped to $\max(0, \cdot)$. For `in`/`not in`, $|L \cap R|$ denotes the count of L-values within the range of R.

op	Satisfied condition	Violated condition	$\lvert T_s \rvert$	$\lvert T_v \rvert$
$>$	$l_{\min} > r_{\max}$	$l_{\max} \leq r_{\min}$	$l_{\max} - r_{\max}$	$r_{\min} - l_{\min} + 1$
$\geq$	$l_{\min} \geq r_{\max}$	$l_{\max} < r_{\min}$	$l_{\max} - r_{\max} + 1$	$r_{\min} - l_{\min}$
$<$	$l_{\max} < r_{\min}$	$l_{\min} \geq r_{\max}$	$r_{\min} - l_{\min}$	$l_{\max} - r_{\max} + 1$
$\leq$	$l_{\max} \leq r_{\min}$	$l_{\min} > r_{\max}$	$r_{\min} - l_{\min} + 1$	$l_{\max} - r_{\max}$
$=$	disjoint $\wedge$ singletons	disjoint	$\lvert L \cap R \rvert$	$n_L - \lvert L \cap R \rvert$
$\neq$	disjoint	disjoint $\wedge$ singletons	$n_L - \lvert L \cap R \rvert$	$\lvert L \cap R \rvert$
`in`	$L \subseteq R$	$L \cap R = \emptyset$	$\lvert L \cap R \rvert$	$n_L - \lvert L \cap R \rvert$
`not in`	$L \cap R = \emptyset$	$L \subseteq R$	$n_L - \lvert L \cap R \rvert$	$\lvert L \cap R \rvert$

- T_s (*satisfied region*): values of L for which l *op* r holds for *all* $r \in R$.
- T_v (*violated region*): values of L for which l *op* r fails for *all* $r \in R$.
- T_w (*warning region*): values of L whose truth value depends on the actual value of R.

Table 3 defines the early-return conditions and region sizes for each operator, where $n_L = l_{\max} - l_{\min} + 1$ and $n_R = r_{\max} - r_{\min} + 1$ denote the number of integer values in each interval.

For the comparison operators ($>$, $\geq$, $<$, $\leq$), the warning region T_w contains values of L whose comparison result depends on the actual value of R. Since Solidity operates on integers, for each $l \in T_w$ the number of R-values satisfying l *op* r forms an arithmetic sequence across the warning region. The expected true count E_w is therefore computed

Algorithm 1 Interval Comparison with Risk Score

Input: Intervals $L = [l_{\min}, l_{\max}]$, $R = [r_{\min}, r_{\max}]$, operator op
Output: ($state$, $risk\_score$)
$n_L \leftarrow l_{\max} - l_{\min} + 1, \quad n_R \leftarrow r_{\max} - r_{\min} + 1$

// Step 1: Early return for definite cases (Table 3)
if SatisfiedCond(op, L, R) **then**
    return (*satisfied*, 0.0)
else if ViolatedCond(op, L, R) **then**
    return (*violated*, 10.0)
end if

// Step 2: Region partitioning (Table 3)
$|T_s|$, $|T_v| \leftarrow$ per-operator formulas from Table 3
$|T_w| \leftarrow n_L - |T_s| - |T_v|$

// Step 3: Compute p_{true}
if $op \in \{$`in`, `not in`$\}$ **then**
    $p_{true} \leftarrow |T_s| \;/\; n_L$
else if $op \in \{=, \neq\}$ **and** $n_R > 1$ **then**
    $p_{true} \leftarrow |T_s| \;/\; (n_L \cdot n_R)$ ▷ = case; ≠: $1 - |T_v|/(n_L \cdot n_R)$
else ▷ $>$, $\geq$, $<$, $\leq$
    $E_w \leftarrow$ expected true count from T_w via arithmetic series over R
    $p_{true} \leftarrow (|T_s| + E_w) \;/\; n_L$
end if

// Step 4: Risk score
$risk\_type \leftarrow$ ClassifyRisk($false\_regions$, op)
$risk\_score \leftarrow base(risk\_type) + (1 - p_{true}) \times span(risk\_type)$
return ($warning$, $risk\_score$)

in closed form as $E_w = (a + b) \cdot |T_w| \;/\; (2 \cdot n_R)$, where a and b are the number of satisfying R-values for the smallest and largest elements of T_w, respectively.

For equality and inequality operators ($=, \neq$), each overlapping L-value has a $1/n_R$ probability of matching any particular R-value, rather than being a definite match. The probability is therefore adjusted by the factor $1/n_R$.

Compound Clauses and Implication. Multiple clauses can be combined using logical operators. The engine evaluates each clause independently and combines the three-valued results according to Table **??**.

4 Evaluation

4.1 Research Questions

We evaluate IntentChecker through the following research questions:

Table 4: Three-valued logic combination rules for compound clauses. S = *satisfied*, V = *violated*, W = *warning*.

`&&`	S	V	W
S	S	V	W
V	V	V	V
W	W	V	W

`||`	S	V	W
S	S	S	S
V	S	V	W
W	S	W	W

`=>`	S	V	W
S	S	V	W
V	S	S	S
W	W	W	W

- **RQ1 (Mitigation)**: Can IntentChecker mitigate real-world numeric logic errors from the Web3Bugs and Numscout datasets (127 bugs) when annotations are provided based on bug report descriptions? We also analyze coverage across the 9 error patterns identified in Section 2.
- **RQ2 (Warning Usefulness)**: When IntentChecker reports `Warning`, does the `false_regions` information help developers identify the conditions under which bugs occur?
- **RQ3 (Expressiveness)**: Can the intent model express developer intentions for general integer/unsigned integer statements? We evaluate this using Code4rena bug reports' expected behavior descriptions as ground truth for developer intent.
- **RQ4 (Developer Perception)**: Do developers find the intent annotation model intuitive and expressive for specifying their intended numeric behaviors?

Soundness Bounded by Debug Annotation Scope. IntentChecker performs sound analysis within the value ranges specified by debug annotations. When developers define variable ranges through `@StateVar` and `@LocalVar` annotations, the validation engine guarantees that if an intent is reported as *Satisfied*, the specified relationship holds for all values within those ranges. However, this soundness guarantee does not extend beyond the specified ranges. If a bug manifests only with input values outside the debug annotation scope, IntentChecker will not detect it. This is an inherent characteristic of debugging tools: just as traditional debuggers only test the specific inputs a developer provides, IntentChecker validates intentions only within the ranges developers specify. Developers are encouraged to use broad, representative ranges during debugging and to iteratively refine their debug annotations when *Warning* results indicate potential issues.

Inline Assembly Not Supported. IntentChecker does not analyze Solidity inline assembly blocks (`assembly { ... }`). When the analyzer encounters inline assembly, it conservatively treats affected variables as having unknown values (top element in the interval lattice). This limitation affects contracts that rely heavily on low-level optimizations or direct EVM manipulation. However, most standard DeFi operations—token transfers, balance updates, fee calculations—are implemented in high-level Solidity and are fully supported. Future work may extend the analyzer to interpret common assembly patterns, particularly those generated by the Solidity compiler for optimization purposes.

6 Related Works

6.1 Numeric Logic Error Detection in Solidity

Logic error detection focuses on identifying discrepancies between developer intentions and actual program behavior in smart contracts.

(Eshghie et al., 2024) addresses business logic violations in smart contracts by using Dynamic Condition Response (DCR) graphs to model expected behaviors including temporal constraints, role-based access control, and inter-function dependencies. The system monitors runtime transactions against user-provided DCR models and their

function mappings, generating violation reports when execution activities deviate from the specified business logic patterns.

(Stephens et al., 2021) verifies temporal properties of smart contracts by accepting user-specified Linear Temporal Logic (LTL) formulas and attack models to check both safety and liveness properties. The system not only detects violations but also generates concrete attack scenarios that demonstrate how the temporal properties can be violated, providing developers with actionable insights into potential logic errors.

(Wei et al., 2024) enables developers to specify function and address behaviors using the ConSol syntax alongside their Solidity code, then verifies whether the implementation adheres to these behavioral specifications and generates behavior-compliant Solidity programs without annotations.

(Wang et al., 2024b) leverages multimodal learning to automatically infer invariants and detect machine unauditable bugs (MUBs) (Zhang et al., 2023) by analyzing both source code and natural language artifacts including comments, function signatures, and documented transaction scenarios. The system employs a fine-tuned model with Tier of Thought (ToT) prompting that progressively predicts transaction contexts, generates and ranks invariants, and ultimately identifies vulnerabilities through three-tier reasoning.

(Zhang, 2024) proposes a novel type system for Solidity variables defined as tuples of ⟨Financial Type, Scaling Factor, Token Unit, Address⟩ to detect accounting errors through type checking. The system reinterprets Solidity code based on detection patterns such as incorrectly adding balances with fees, enabling systematic identification of numeric logic errors that arise from mismatched financial operations.

Furthermore, there are also approaches that address multiple transaction logic errors such as price manipulation (Wu et al., 2023; Wang et al., 2024a; Xi et al., 2024; Arora et al., 2024; Kong et al., 2023) and flash loan attacks (Li et al., 2025; Ramezany et al., 2023).

In contrast to existing approaches, IntentChecker operates within the debugging context to provide immediate feedback on code under development, fundamentally differing from tools that only verify completed code. Through `@During` annotations, it uniquely enables validation of intermediate execution states within functions, allowing developers to verify their step-by-step intentions. Moreover, rather than being limited to specific patterns, IntentChecker comprehensively validates all numeric operations using interval-domain abstract interpretation for sound verification, achieving both generality and reliability simultaneously.

6.2 Specification-based Smart Contract Verification

Specification-based verification approaches focus on verifying general properties and correctness requirements of smart contracts rather than specific developer intentions.

(Consensys Diligence, 2025c) provides a framework that transforms developer annotations into Solidity assertion code, enabling testing with existing tools like MythX (Consensys Diligence, 2025b) and Mythril (Consensys Diligence, 2025a). The framework supports multiple annotation types including if_succeed (conditions for successful function returns), invariant (contract-level properties that must always hold), if_updated (conditions checked when state variables change), and assert (validations

at specific execution points), making formal specifications accessible through familiar testing pipelines.

(Wesley et al., 2022) builds upon Scribble (Consensys Diligence, 2025c) to verify whether Solidity code satisfies specifications written in the Scribble language, leveraging C-based verification tools like SeaHorn (Gurfinkel et al., 2015), libFuzzer (LLVM, 2025), and KLEE (Cadar et al., 2008) by transforming smart contracts into LLVM-IR based harnesses. The framework abstracts the 2^{160} possible user addresses into a small set of representative users (implicit, transient, and explicit), enabling verification of properties that must hold for arbitrary numbers of users, such as accounting properties like "the contract balance equals the sum of all user deposits."

(Nguyen et al., 2023) verifies Solidity code using two types of annotations: specification annotations (loop invariants, call invariants, and call modifies) that developers declare as assumptions, and verification annotations (ensures, reverts_if, and contract invariants) that the tool checks against the implementation. The system assumes the truth of specification annotations and uses them to verify whether the code satisfies function post-conditions, correctly handles revert conditions, and maintains contract invariants throughout execution.

(Bartoletti et al., 2024) focuses on verifying liquidity properties of smart contracts through formal methods by converting both the contract code and user-defined liquidity properties into SMT constraints. The system then checks whether the contract satisfies these properties using SMT solvers, generating counterexamples when violations are detected to help developers understand potential liquidity issues.

While specification-based approaches focus on verifying high-level contract properties and invariants in completed code, IntentChecker addresses a fundamentally different problem: validating specific developer intentions about numeric values during the debugging process. Unlike existing tools that treat library functions as black boxes, IntentChecker performs interval-domain analysis that tracks actual value ranges through library function calls (e.g., SafeMath operations), enabling developers to verify their step-by-step numeric intentions rather than abstract contract-level properties.

6.3 Tools for Strengthening Security During Development Phase

Development-phase security tools integrate vulnerability detection directly into the coding workflow, enabling developers to identify and address security issues as they write code rather than in post-development analysis. (Ren et al., 2021) presents an integrated development environment for smart contract development that combines natural language model-based code autocompletion with security validation. The platform integrates existing static analysis tools including Oyente (Luu et al., 2016), Mythril (Consensys Diligence, 2025a), and Securify (Tsankov et al., 2018) to provide comprehensive security reports during development workflow.

(Protofire, 2025) provides a Solidity linter that detects predefined vulnerability patterns including low-level calls, reentrancy, and deprecated function usage during development. The tool enforces security best practices by checking for patterns such as proper visibility declarations, send result validation, and secure compiler version usage.

While these development-phase tools rely on predefined detection patterns for known vulnerabilities, IntentChecker enables developers to specify their intended numeric behaviors through annotations and validates whether the actual program execution matches these specified intentions.

7 Conclusion

References

Arora, S., et al.: SecPLF: Secure protocols for loanable funds against oracle manipulation attacks. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIACCS), pp. 1394–1405 (2024). https://doi.org/10.1145/3634737.3637681

Bartoletti, M., et al.: Solvent: liquidity verification of smart contracts. In: International Conference on Integrated Formal Methods (iFM), pp. 256–266 (2024). https://doi.org/10.1007/978-3-031-76554-4$_1$4

Cadar, C., et al.: KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In: OSDI, pp. 209–224 (2008). https://doi.org/10.5555/1855741.1855756

Chaliasos, S., et al.: Smart contract and defi security tools: Do they meet the needs of practitioners?. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1–13 (2024). https://doi.org/10.1145/3597503.3623302

Chen, J., et al.: NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. IEEE Transactions on Software Engineering (2025). https://doi.org/10.1109/TSE.2025.3555622

Consensys Diligence: Mythril. https://github.com/ConsenSysDiligence/mythril (2025). Accessed December 2025

Consensys Diligence: MythX. https://mythx.io/ (2025). Accessed December 2025

Consensys Diligence: Scribble. https://diligence.security/scribble/ (2025). Accessed January 2026

Eshghie, M., et al.: Highguard: Cross-chain business logic monitoring of smart contracts. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 2378–2381 (2024). https://doi.org/10.1145/3691620.3695356

Gurfinkel, A., et al.: The SeaHorn verification framework. In: International Conference on Computer Aided Verification, pp. 343–361 (2015). https://doi.org/10.1007/978-3-319-21690-4$_2$0

Jeon, I., et al.: SolQDebug: Debug Solidity Quickly for Interactive Immediacy in Smart Contract Development (2025). https://doi.org/10.21203/rs.3.rs-8077153/v1

Kong, Q., et al.: Defitainter: Detecting price manipulation vulnerabilities in defi protocols. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 1144–1156 (2023). https://doi.org/10.1145/3597926.3598124

Li, X., et al.: Penetrating the hostile: Detecting defi protocol exploits through cross-contract analysis. IEEE Transactions on Information Forensics and Security 20, 11759–11774 (2025). https://doi.org/10.1109/TIFS.2025.3626979

LLVM: libFuzzer. https://llvm.org/docs/LibFuzzer.html (2025). Accessed December 2025

Luu, L., et al.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 254–269 (2016). https://doi.org/10.1145/2976749.2978309

Nguyen, T.D., et al.: An idealist's approach for smart contract correctness. In: International Conference on Formal Engineering Methods, pp. 11–28 (2023). https://doi.org/10.1007/978-981-99-7584-6$_2$

Protofire: Solhint. https://github.com/protofire/solhint (2025). Accessed Janauary 2026

Ramezany, S., Setthawong, R., Setthawong, P.: Midnight: An Efficient Event-driven EVM Transaction Security Monitoring Approach For Flash Loan Detection. In: Proceedings of the 2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE), pp. 237–241 (2023). https://doi.org/10.1109/JCSSE58229.2023.10201970

Ren, M., et al.: Making smart contract development more secure and easier. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), pp. 1360–1370 (2021). https://doi.org/10.1145/3468264.3473929

Soud, M., Liebel, G., Hamdaqa, M.: A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. Empirical Software Engineering 29(1), 13 (2024). https://doi.org/10.1007/s10664-023-10398-5

Stephens, J., et al.: SmartPulse: automated checking of temporal properties in smart contracts. In: Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP), pp. 555–571 (2021). https://doi.org/10.1109/SP40001.2021.00085

Sun, Y., et al.: Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). https://doi.org/10.1145/3597503.3639117

Tsankov, P., et al.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 67–82 (2018). https://doi.org/10.1145/3243734.3243780

Wang, D., et al.: Defiguard: A price manipulation detection service in defi using graph neural networks. IEEE Transactions on Services Computing (2024). https://doi.org/10.1109/TSC.2024.3489439

Wang, S.J., Pei, K., Yang, J.: Smartinv: Multimodal learning for smart contract invariant inference. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2217–2235 (2024). https://doi.org/10.1109/SP54263.2024.00126

Wei, G., et al.: Consolidating Smart Contracts with Behavioral Contracts. Proceedings of the ACM on Programming Languages 8(PLDI), 965–989 (2024). https://doi.org/10.1145/3656416

Wesley, S., et al.: Verifying Solidity smart contracts via communication abstraction in SmartACE. In: International Conference on Verification, Model Checking, and Abstract Interpretation, pp. 425–449 (2022). https://doi.org/10.1007/978-3-030-94583-1$_2$1

Wu, S., et al.: Defiranger: Detecting defi price manipulation attacks. IEEE Transactions on Dependable and Secure Computing 21(4), 4147–4161 (2023). https://doi.org/10.1109/TDSC.2023.3346888

Xi, R., Wang, Z., Pattabiraman, K.: POMABuster: Detecting Price Oracle Manipulation Attacks in Decentralized Finance. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 3923–3942 (2024). https://doi.org/10.1109/SP54263.2024.00257

Zhang, B.: Towards finding accounting errors in smart contracts. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). https://doi.org/10.1145/3597503.3639128

Zhang, W., et al.: Nyx: Detecting exploitable front-running vulnerabilities in smart contracts. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2198–2216 (2024). https://doi.org/10.1109/SP54263.2024.00146

Yang, S., et al.: Hyperion: Unveiling DApp Inconsistencies Using LLM and Dataflow-Guided Symbolic Execution. In: Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2125–2137 (2025). https://doi.org/10.1109/ICSE55347.2025.00015

Zhang, Z., et al.: Demystifying exploitable bugs in smart contracts. In: Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 615–627 (2023). https://doi.org/10.1109/ICSE48619.2023.00061

Zheng, Z., et al.: Dappscan: building large-scale datasets for smart contract weaknesses in dApp projects. IEEE Transactions on Software Engineering 50(6), 1360–1373 (2024). https://doi.org/10.1109/TSE.2024.3383422

Zhou, L., et al.: Sok: Decentralized finance (defi) attacks. In: Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP), pp. 2444–2461 (2023). https://doi.org/10.1109/SP46215.2023.10179435